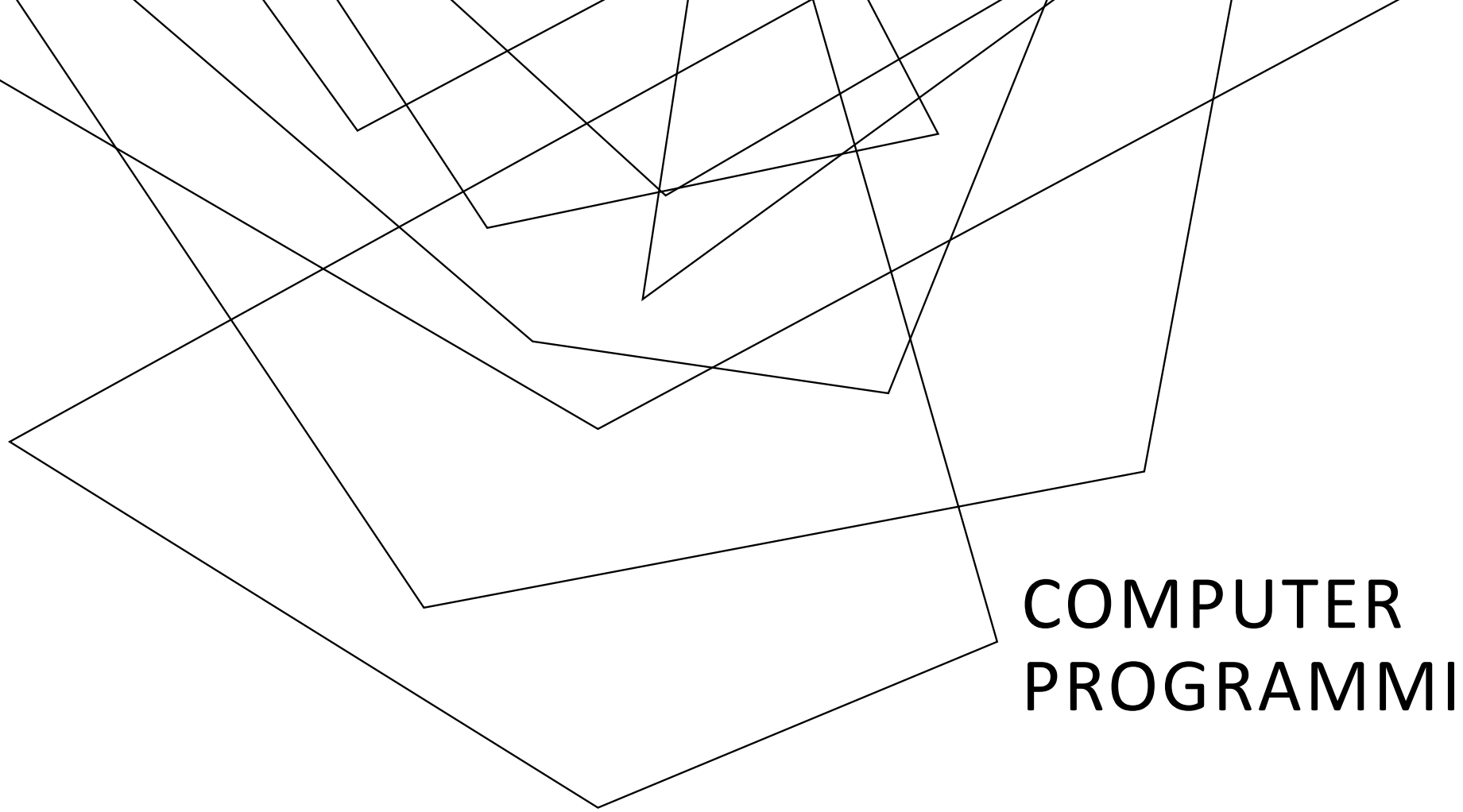


COMPUTER PROGRAMMING 1

DR. SAMERA OBAID BARRAOOD

INTRODUCTION TO PROGRAMMING LANGUAGE

PROGRAM

A program is a set of instruction gave to the computer to execute successive operations leads to solve specific problem. In general, to solve any problem in computer we have to follow these steps:

1. **Analyze the problem.**
2. **Write an Algorithm.**
3. **Draw flowchart.**
4. **Convert the flowchart to program.**
5. **Run the program and test the solution.**

PROGRAMMING LANGUAGE GENERATIONS

First-generation: Machine language

Second-generation: Assembly language

Third-generation: High-level language

Fourth-generation

Fifth-generation

1GL: MACHINE LANGUAGE

•**Description**: Also known as machine language, this is the lowest level of programming language directly understood by a computer's CPU.

•**Characteristics**:

- Consists of binary code (0s and 1s).
- Specific to a particular type of hardware.

•**Example**: Direct binary instructions for a CPU.

2GL: ASSEMBLY LANGUAGE

•**Description**: Known as assembly language, it uses mnemonic codes and symbols to represent machine-level instructions.

•**Characteristics**:

- More human-readable than machine language.
- Still hardware-specific, requiring an assembler to convert to machine code.

•**Example**

```
MOV AX, 1
ADD AX, 2
```

3GL: HIGH-LEVEL LANGUAGE

- **Description**: These are high-level programming languages that allow for more abstraction from hardware.
- **Characteristics**:
  - More user-friendly, closer to human languages.
  - Portable across different computer systems.
  - Include structured programming concepts.
- **Examples**:
  - C
  - Java
  - Python
  - Ruby

4GL : FOURTH GENERATION LANGUAGE

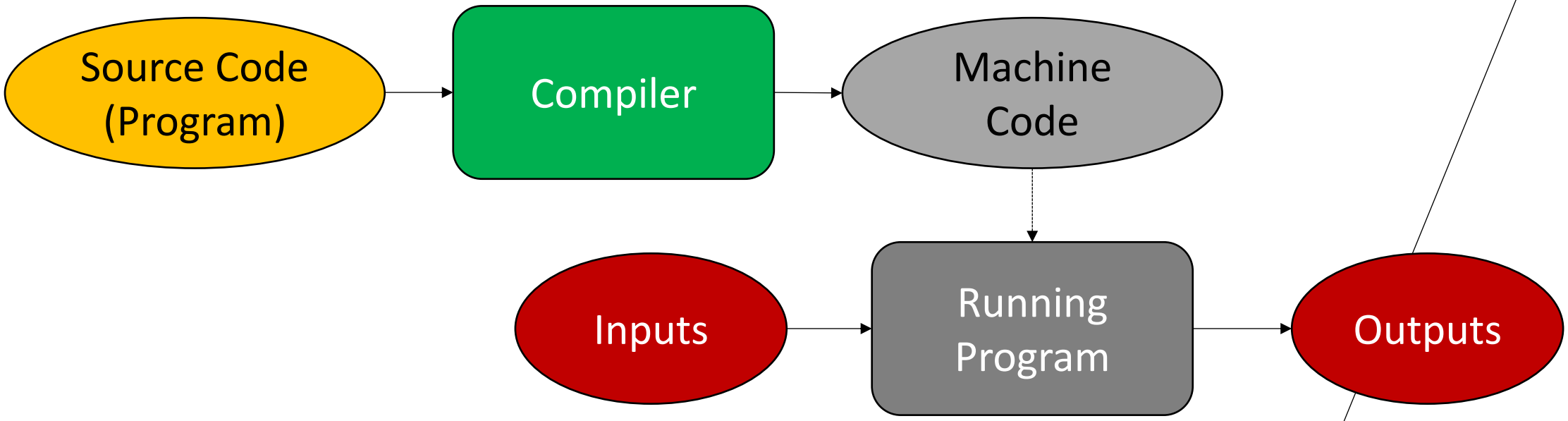
- **Description:**
  - These languages are designed to be even closer to human language and are often used for database manipulation and report generation.
- **Characteristics:**
  - Focus on reducing the complexity of programming.
  - Often domain-specific and more productive for certain types of tasks.
- **Examples:**
  - SQL (Structured Query Language)
  - MATLAB
  - R

5GL: FIFTH GENERATION LANGUAGE

- **Description:**
  - These languages are used for artificial intelligence and logic programming.
- **Characteristics:**
  - Focus on solving problems through constraints and logic rather than explicit programming instructions.
  - Often include natural language processing capabilities.
- **Examples:**
  - Prolog
  - Mercury

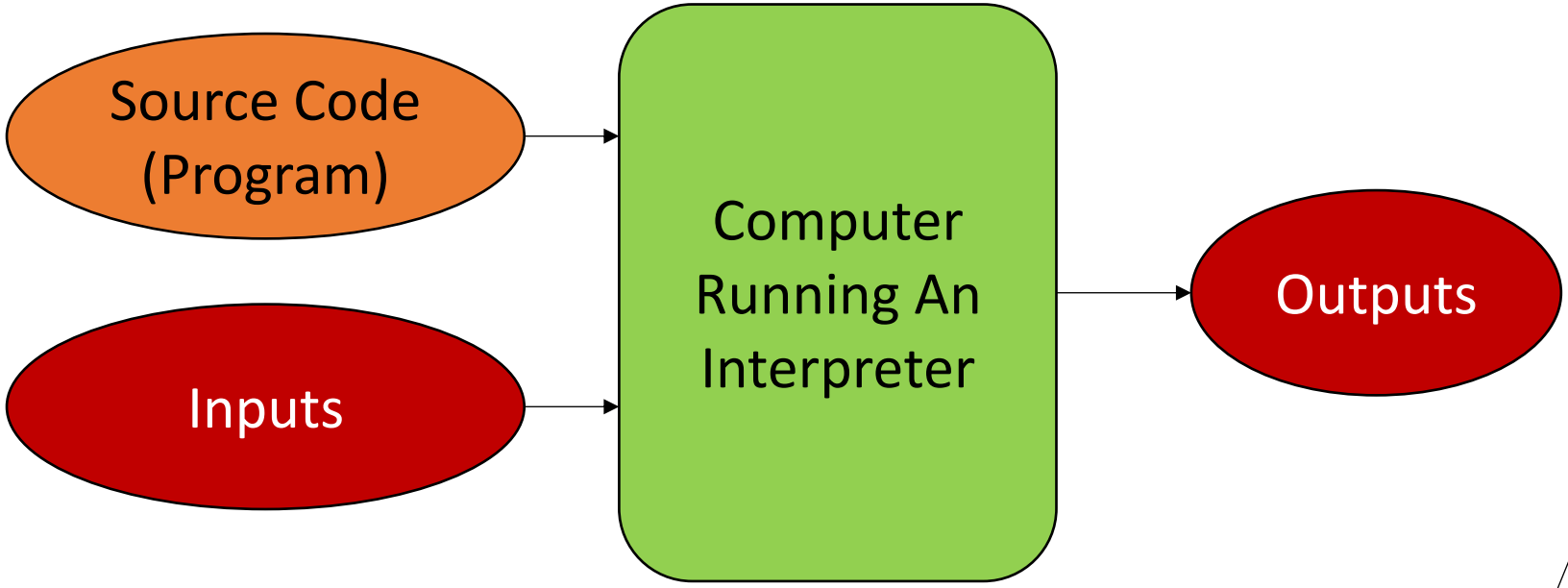
COMPILING A HIGH-LEVEL LANGUAGE

- **A compiler** is a complex software that takes a program written in a high-level language and translates it into an equivalent program in the machine language of some computer



INTERPRETING A HIGH-LEVEL LANGUAGE

- An interpreter is a software that analyzes and executes the source code instruction-by-instruction (on-the-fly) as necessary



- E.g., Python is an interpreted language

COMPILING VS. INTERPRETING

- Compiling is **a static** (i.e., pre-execution), one-shot translation
  - Once a program is compiled, it may be run over and over again without further need for the compiler or the source code
- Interpreting is **dynamic** (i.e., happens during execution)
  - The interpreter and the source code are needed every time the program runs
- Compiled programs tend to be faster, while interpreted ones lend themselves to a more flexible programming environments (*they can be developed and run interactively*)

POPULAR OF PROGRAMMING LANGUAGES

POPULAR OF PROGRAMMING LANGUAGES

1. Python

- **Description**: A versatile, high-level language known for its readability and simplicity.
- **Use Cases**: Data science, web development, automation, artificial intelligence, and scientific computing.

2. JavaScript

- **Description**: A dynamic language primarily used for web development to create interactive web pages.
- **Use Cases**: Front-end and back-end web development (using Node.js), mobile app development, and game development.

3. Java

- **Description**: A widely-used, object-oriented programming language that runs on the Java Virtual Machine (JVM).
- **Use Cases**: Enterprise applications, Android app development, and large-scale systems.

POPULAR OF PROGRAMMING LANGUAGES

4. C#

- **Description**: A language developed by Microsoft, often used for building Windows applications.
- **Use Cases**: Game development (using Unity), enterprise applications, and web development (using ASP.NET).

5. C++

- **Description**: An extension of C that includes object-oriented features. Known for its performance and control over system resources.
- **Use Cases**: System/software development, game development, and applications requiring high performance.

6. Ruby

- **Description**: A dynamic, object-oriented language known for its simplicity and productivity.
- **Use Cases**: Web development (especially with the Ruby on Rails framework), automation, and prototyping.

POPULAR OF PROGRAMMING LANGUAGES

7. PHP

- **Description**: A server-side scripting language designed for web development but also used as a general-purpose language.
- **Use Cases**: Web development, content management systems (like WordPress), and server-side applications.

8. Swift

- **Description**: A language developed by Apple for iOS and macOS application development.
- **Use Cases**: Mobile app development for iPhone and iPad, as well as software for macOS.

9. Go (Golang)

- **Description**: A statically typed language developed by Google, known for its simplicity and efficiency.
- **Use Cases**: Cloud services, distributed systems, and microservices.

10. Kotlin

- **Description**: A modern programming language that runs on the JVM and is fully interoperable with Java.
- **Use Cases**: Android app development and server-side applications.

IMPORTANCE OF PROGRAMMING LANGUAGE

IMPORTANCE OF PROGRAMMING LANGUAGE

1. Problem Solving

- **Logical Thinking**: Programming languages enable individuals to formulate solutions to complex problems through logical reasoning and structured thinking.
- **Automation**: They allow for automating repetitive tasks, improving efficiency and productivity.

2. Development of Applications

- **Software Creation**: Programming languages are essential for developing software applications, from simple scripts to complex systems.
- **Web Development**: Languages like HTML, CSS, JavaScript, and PHP are fundamental in building and maintaining websites and web applications.

IMPORTANCE OF PROGRAMMING LANGUAGE

3. Data Analysis and Visualization

- **Data Manipulation**: Languages such as Python and R are vital for data analysis, enabling users to manipulate and visualize data effectively.
- **Machine Learning and AI**: Many programming languages provide libraries and frameworks that facilitate the development of machine learning models and AI applications.

4. Career Opportunities

- **Job Market Demand**: Proficiency in programming languages is highly sought after across various industries, offering numerous career paths, including software development, data science, and cybersecurity.
- **Skill Development**: Learning programming languages enhances critical thinking and problem-solving skills, which are valuable in any professional setting.

IMPORTANCE OF PROGRAMMING LANGUAGE

5. Innovation and Technology Advancement

- **Driving Innovation**: Programming languages are at the core of technological advancements, enabling the development of new applications, tools, and systems.
- **Research and Development**: They play a crucial role in scientific research, allowing researchers to implement algorithms and simulations.

6. Cross-Disciplinary Applications

- **Interdisciplinary Use**: Programming languages are used in various fields, including finance, healthcare, education, and entertainment, demonstrating their versatility and importance.
- **Collaboration**: Knowledge of programming fosters collaboration between disciplines, such as data scientists working with domain experts to develop solutions.

IMPORTANCE OF PROGRAMMING LANGUAGE

7. Community and Open Source

- **Support and Resources**: Programming languages often have large communities that provide support, resources, and libraries, making it easier to learn and solve problems.
- **Open Source Contributions**: Many programming languages support open-source projects, allowing individuals to contribute to and learn from real-world applications.

INTRODUCTION TO ALGORITHMS AND FLOWCHARTS

ALGORITHM

An **algorithm** in computer science is a step-by-step procedure or formula for solving a problem or performing a task.

It consists of a finite sequence of well-defined instructions that take an input, process it, and produce an output.

CONDITIONS FOR AN ALGORITHM

1.Finiteness:

The algorithm must terminate after a finite number of steps. It should not run indefinitely.

2.Definiteness:

Each step of the algorithm must be precisely defined and unambiguous. The operations to be performed should be clear.

3.Input:

An algorithm can have zero or more inputs. These inputs are the values that the algorithm processes to produce an output.

4.Output:

An algorithm must produce one or more outputs. These outputs are the results generated after processing the inputs.

5.Effectiveness:

The steps of the algorithm must be basic enough that they can be performed by a person (or a computer) using a pencil and paper. Each operation should be feasible.

FLOWCHART

Flowcharts are graphs that represent the formal view used to solve any problem.

Flowcharts help the programmer to write his program.

Flowcharts consist of a shapes connected by a straight lines.

The following table shows these shapes and their operations.



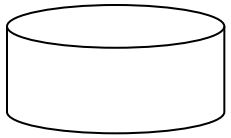
Terminal symbol - indicates the beginning and end points of an algorithm.



Process symbol - shows an instruction other than input, output or selection.



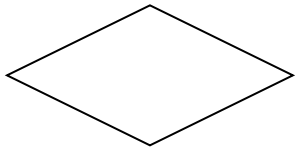
Input-output symbol - shows an input or an output operation.



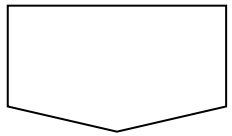
Disk storage I/O symbol - indicates input from or output to disk storage.



Printer output symbol - shows hardcopy printer output.



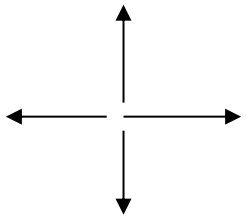
Selection symbol - shows a selection process for two-way selection.



Off-page connector - provides continuation of a logical path on another page.



On-page connector - provides continuation of logical path at another point in the same page.



Flow lines - indicate the logical sequence of execution steps in the algorithm.

ALGORITHMS AND FLOWCHARTS

We can summarize the use of algorithms and flowcharts as follow:

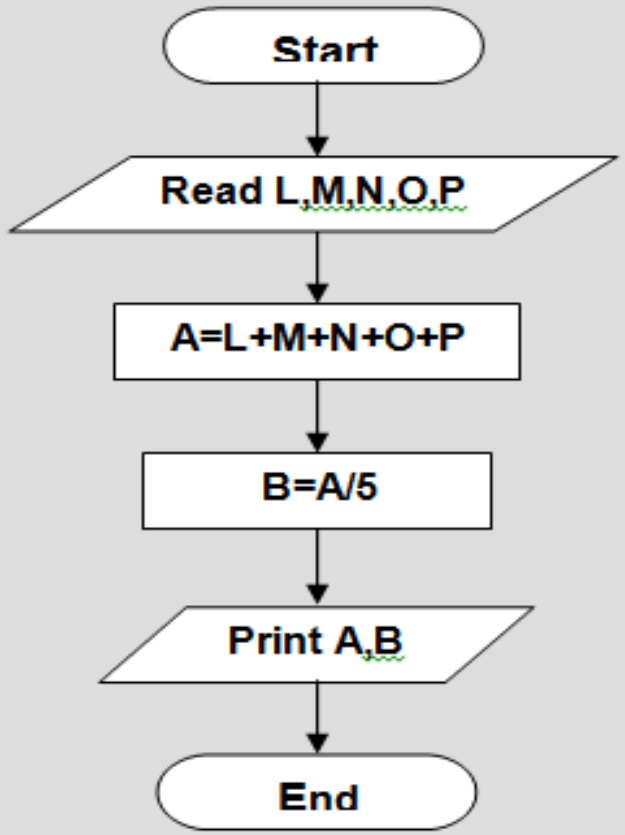
1. **To show the mathematical logic used to solve problems.**
2. **To show how the data processing is done.**
3. **Helps the programmer to write his program.**
4. **Divides the big problem to smaller parts.**
5. **To avoid the errors that occurred during writing the program.**
6. **It is a middle step between problem difficulty and its conversion to suitable program.**
7. **Easy to convert it to any programming language.**

Example-1:

Write an algorithm and draw a flow chart to read five numbers and find their sum and average. Print the results.

Solution:

1. Start
2. Read L, M, N, O, P
3. Find Sum (A) , A=L+M+N+O+P
4. Find Average (B) , B= A / 5
5. Print A, B
6. End



Example-2:

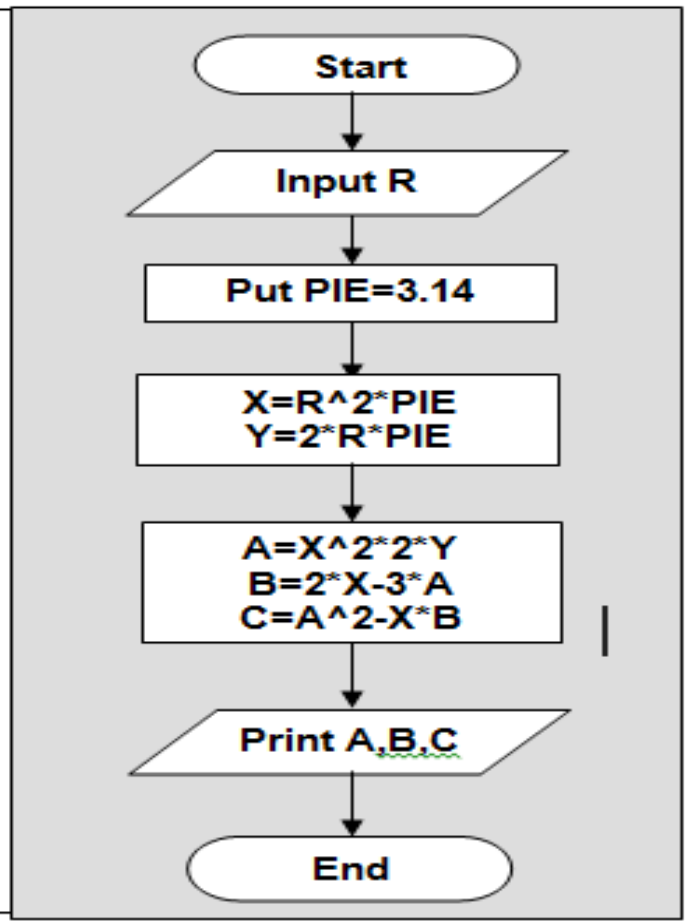
Write an algorithm and draw a flow chart to find the value of A, B, C from the following equations:

$$A = X^2+2Y \quad , \quad B = 2X-3A \quad , \quad C = A^2-XB$$

Where X and Y represents a circle area and circumference respectively. Input the radius (R) and print the value of A, B and C.

Solution:

1. Start
2. Input Radius (R)
3. Put PIE = 3.14
4. Find Area (X), X= R^2*PIE
5. Find Circumference (Y), Y= 2*R*PIE
6. Find A, A= X^2+2*Y
7. Find B, B=2*X-3*A
8. Find C, C=A^2-X*B
9. Print A, B, C
10. End



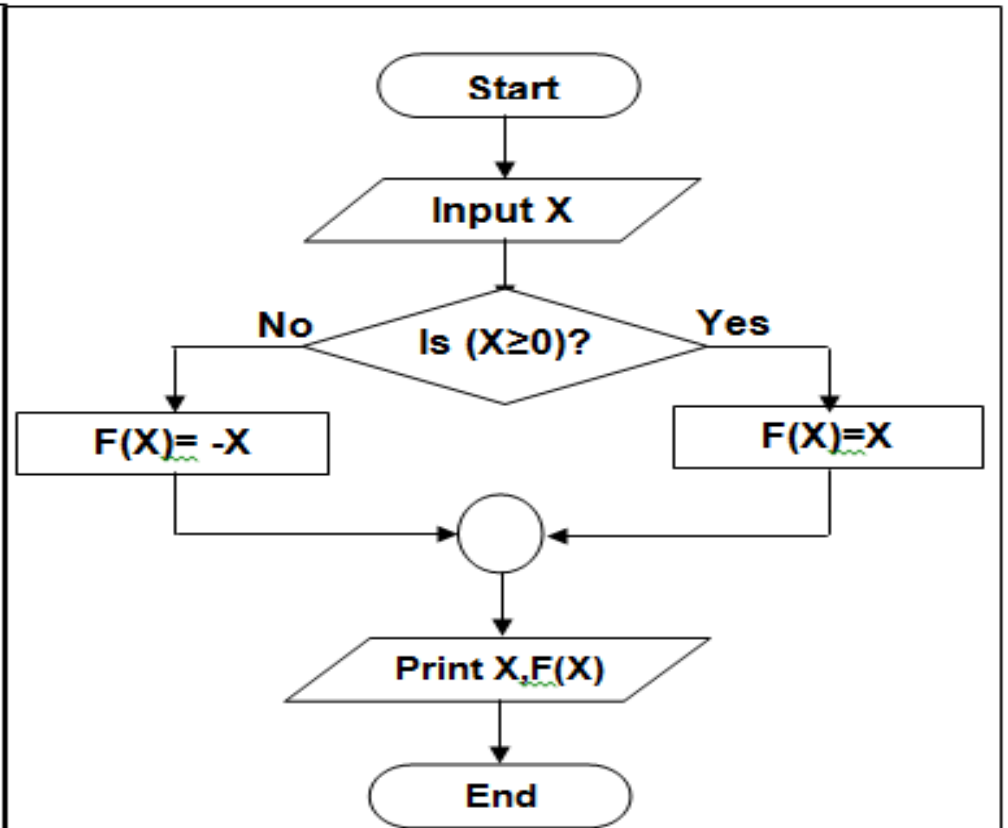
Example-3

Write an algorithm and draw a flow chart to find the value of the below function. Input X and print F(X) to each value of X.

$$F(X)=\begin{cases} X & : \text{if } X \geq 0 \\ -X & : \text{if } X < 0 \end{cases}$$

Solution

1. Start
2. Input x
3. Is $x \geq 0$?
   if "Yes" then continue.
   if "No" then go to step-5.
4. Find F(x), F(x) = x : Goto step-6
5. Find F(x), F(x) = -x.
6. Print x , F(x)
7. End



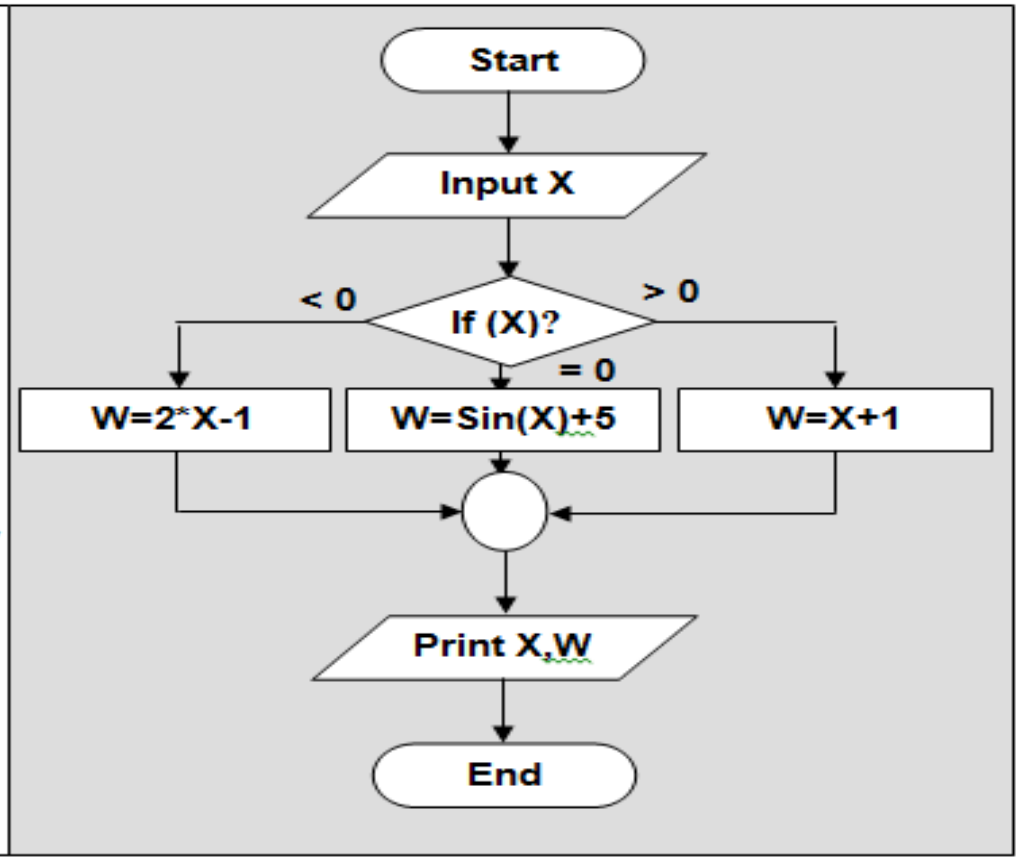
Example-4

Write an algorithm and draw a flow chart to evaluate W from the below equation. Input X and print the value of W for each value of X.

$$W = \begin{cases} X+1 & : X > 0 \\ Sin(X)+5 & : X = 0 \\ 2X-1 & : X < 0 \end{cases}$$

Solution:

1. Start
2. Input x
3. If x > 0 then continue
   If x = 0 then goto step-5
   If x < 0 then goto step-6
4. Find W, W = X+1 : Goto step-7
5. Find W, W = Sin(X)+5: Goto step-7
6. Find W, W = 2*X-1
7. Print X , W
8. End

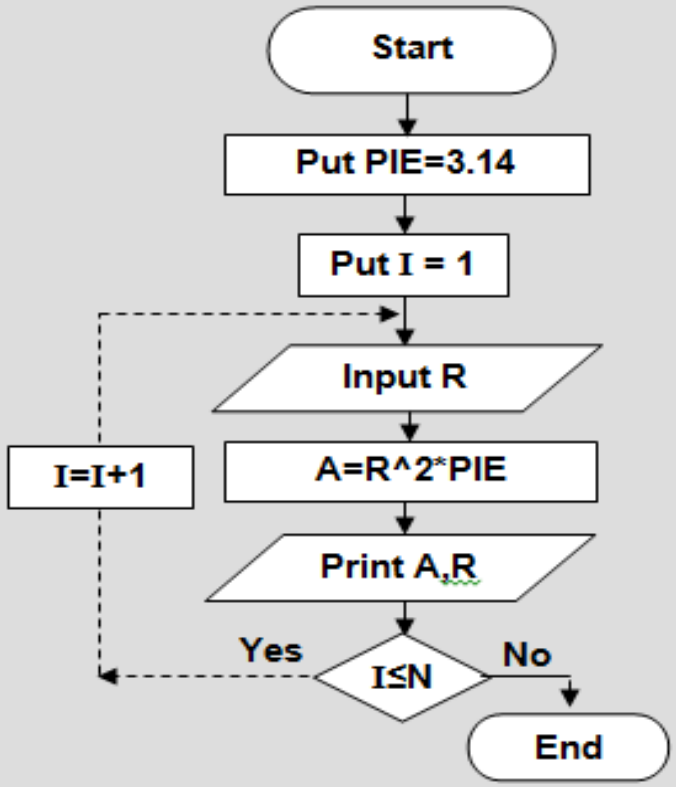


Example-5

Write an algorithm and draw a flow chart to find the area of (N) circles. Input the circles and print the result. Use the general form.

Solution:

1. Start
2. Put PIE = 3.14
3. Put I = 1
4. Input Radius (R)
5. Find Area (A), A=R^2*PIE
6. Print R , A
7. Is I ≤ N ?
   If "Yes" Then I=I+1 :Goto step-4
   If "No" Then continue
8. End



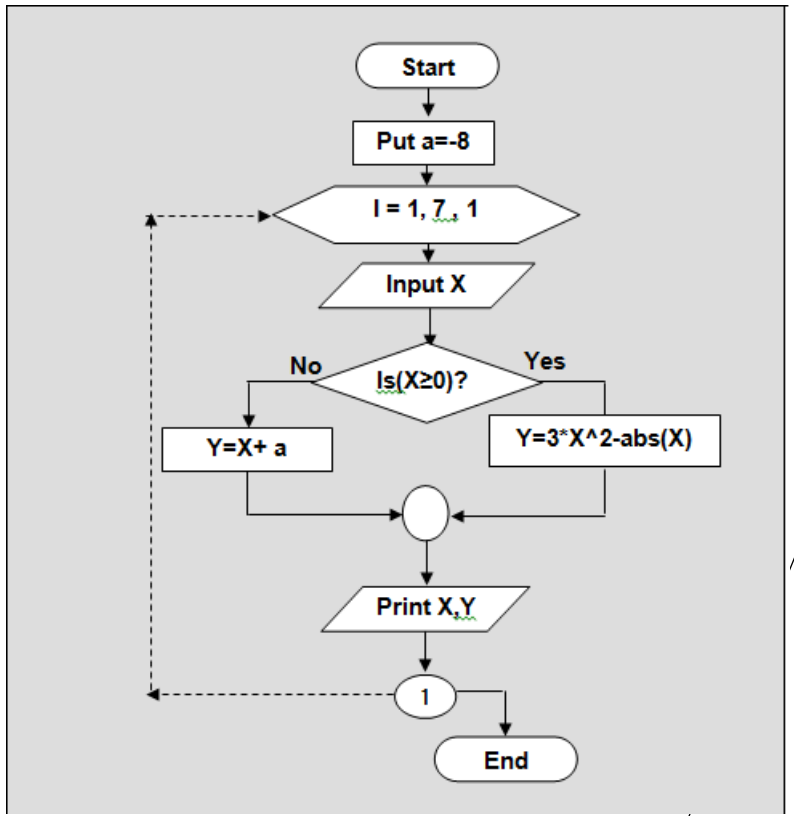
Example-6

Write an algorithm and draw a flow chart to evaluate Y from the bellow equations for seven entering values of X. If you know that a=-8, print the value of Y for each value of X. Use the conventional form.

$$Y = \begin{cases} 3X^2 - |X| & : X \geq 0 \\ X + a & : X < 0 \end{cases}$$

Solution:

1. Start
2. Put a = -8
3. For I=1 To 7 step 1
4. Input X
5. Is X≥0 ?
   If "Yes" then continue
   If "No" : then goto Step-7
6. Find Y, Y=3*X^2-abs(X) : Goto step-8
7. Find Y, Y= X+a
8. Print X,Y
9. Next I
10. End

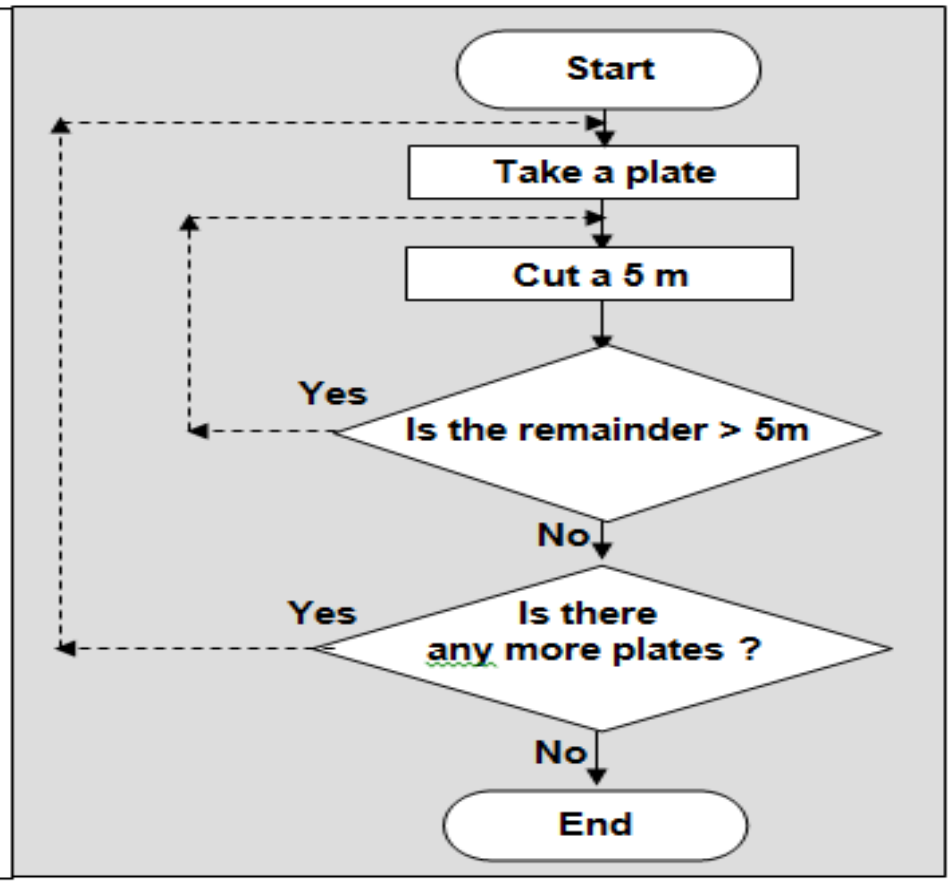


Example-7

Metal plate cutter machine used to cut plates its length is (5 meters). If you know that the length of each plate is more than (30 meters), write an algorithm and draw a flowchart to clarify this operation.

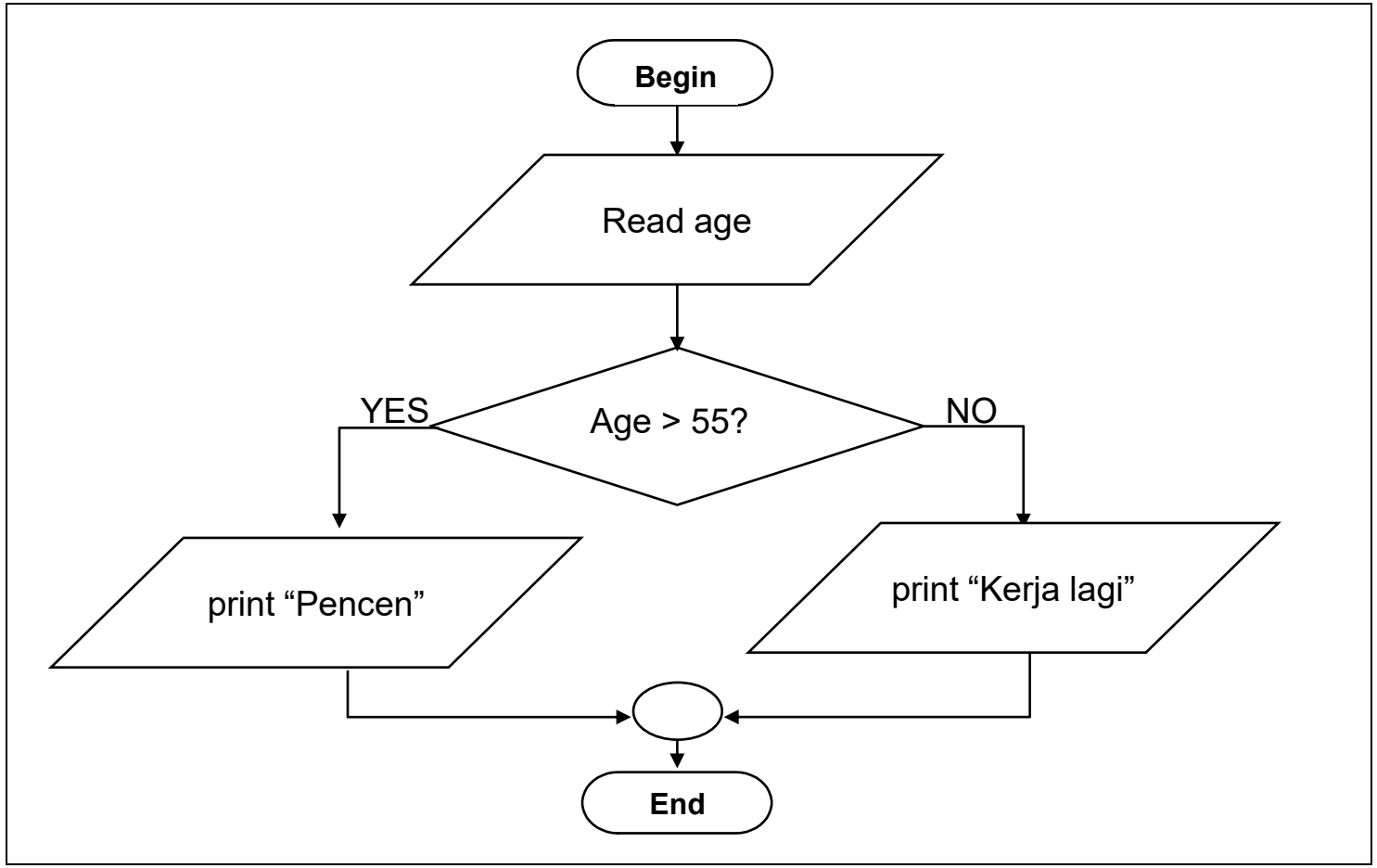
Solution:

1. Start
2. Take a metal plate.
3. Cut a 5m.
4. Is the remainder > 5m
   If "Yes" then goto step-3
   If "No" then continue
5. Is there any more plates ?
   If "Yes" then goto step-2
   If "No" then continue
6. End



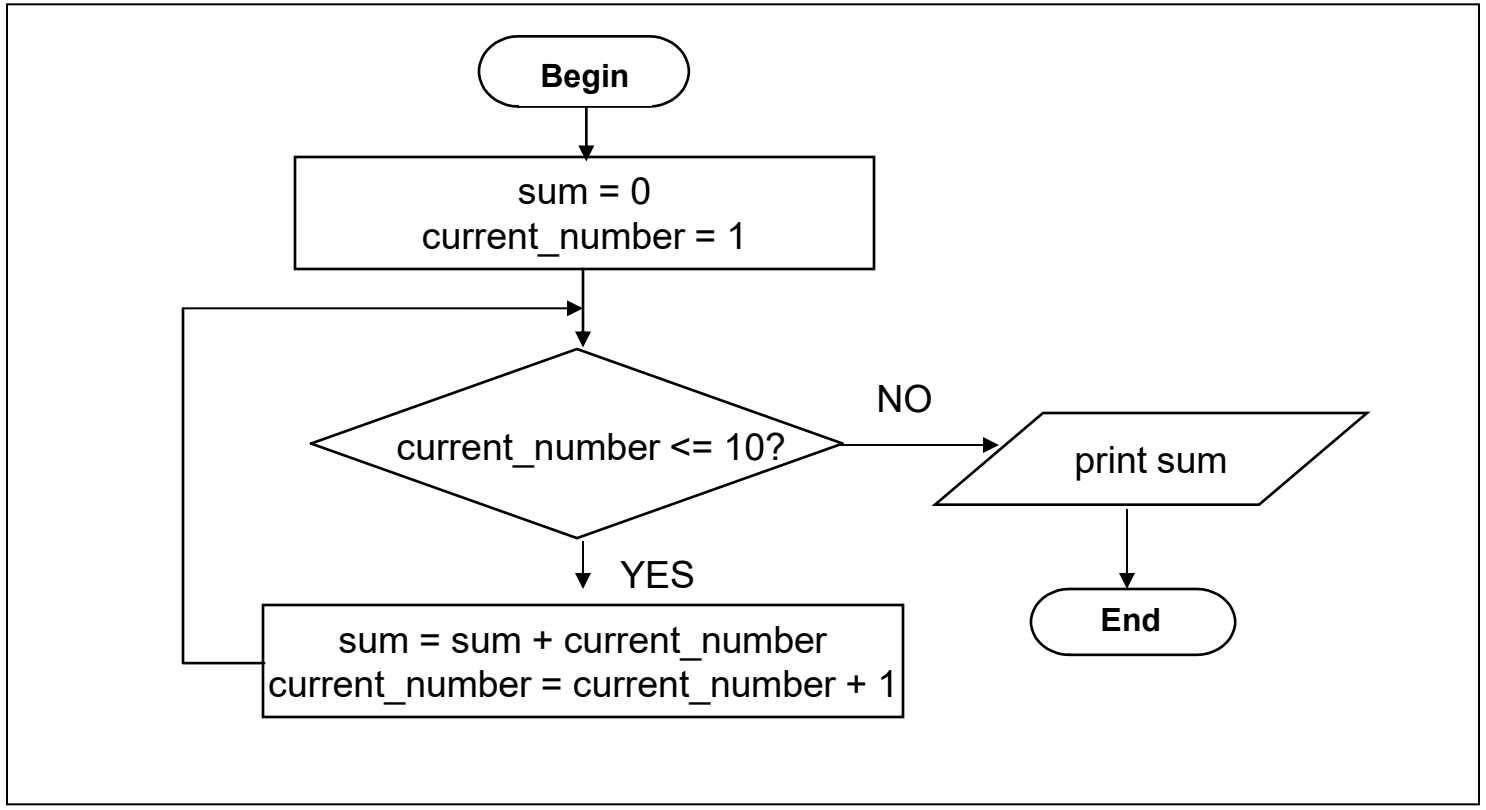
EXERCISES: ALGORITHM & FLOWCHART

What does this flowchart do?



EXERCISES: ALGORITHM & FLOWCHART

What does this flowchart do?



EXERCISES: ALGORITHM & FLOWCHART

- Write an algorithm and draw a flowchart that will calculate the roots of a quadratic equation

$$ax^2 + bx + c = 0$$

- Hint: **d** = sqrt ($b^2 - 4ac$), and the roots are: ***x*1** = (–*b* + *d*)/2*a* and ***x*2** = (–*b* – *d*)/2*a*

EXERCISES: ALGORITHM & FLOWCHART

1.) Create an algorithm and a flowchart that will accept/read two numbers and then display the bigger number.

EXERCISES: ALGORITHM & FLOWCHART

2.) Create an algorithm and a flowchart that will compute the area of a circle.

EXERCISES: ALGORITHM & FLOWCHART

3.) Create an algorithm and a flowchart that will compute the sum of two numbers. If the sum is below or equal to twenty, two numbers will be entered again. If the sum is above 20, it will display the sum.

LAB ACTIVITY: ALGORITHM & FLOWCHART

4) Create an algorithm and a flowchart that will output the largest number among the three numbers.

INTRODUCTION TO PYTHON

WHAT IS PYTHON?

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python emphasizes code clarity and allows programmers to express concepts in fewer lines of code compared to other languages.

KEY FEATURES OF PYTHON

1. **Easy to Learn and Use**:

   Python has a simple syntax that resembles natural language, making it accessible for beginners.

2. **Interpreted Language**:

   Python code is executed line by line, which makes debugging easier and allows for immediate feedback during development.

3. **Versatile**:

   Python is a general-purpose language, meaning it can be used for various applications, including web development, data analysis, artificial intelligence, scientific computing, automation, and more.

KEY FEATURES OF PYTHON

4. Extensive Libraries and Frameworks:

1. Python has a rich ecosystem of libraries and frameworks (e.g., NumPy, Pandas, Matplotlib, TensorFlow, Flask) that extend its functionality and simplify complex tasks.

5. Cross-Platform Compatibility:

1. Python can run on various operating systems, including Windows, macOS, and Linux, making it highly portable.

6. Strong Community Support:

1. Python has a large and active community, providing extensive resources, documentation, and forums for support.

7. Object-Oriented and Functional Programming:

1. Python supports multiple programming paradigms, including object-oriented, procedural, and functional programming, allowing for flexibility in coding styles.

APPLICATIONS OF PYTHON

- **Web Development**: Frameworks like Django and Flask make it easy to build robust web applications.
- **Data Science and Machine Learning**: Libraries like Pandas, NumPy, and Scikit-learn enable data manipulation, analysis, and machine learning.
- **Automation and Scripting**: Python is widely used for automating tasks and writing scripts to streamline workflows.
- **Game Development**: Libraries such as Pygame allow for the development of games and interactive applications.
- **Scientific Computing**: Python is used in research and scientific computing for simulations and data analysis.

WHY PYTHON

WHY PYTHON?

1. Ease of Learning

- **Readable Syntax**: Python's syntax is straightforward and resembles natural language, making it accessible for beginners.
- **Quick Prototyping**: Students can quickly write and test code, which is crucial for learning and experimentation.

2. Versatility

- **General-Purpose Language**: Python is not limited to AI; it can be used for web development, automation, scientific computing, and more.
- **Cross-Platform Compatibility**: Python programs can run on various operating systems without modification.

3. Rich Ecosystem of Libraries

- **Numerous Libraries for AI**:
  - **NumPy**: For numerical computations.
  - **Pandas**: For data manipulation and analysis.
  - **Matplotlib**: For data visualization.
  - **Scikit-learn**: For machine learning algorithms.
  - **TensorFlow and PyTorch**: For deep learning applications.
- These libraries simplify complex tasks and enable students to focus on learning concepts rather than low-level coding.

4. Community and Support

- **Large Community**: Python has a vast and active community, providing extensive resources, documentation, and forums for support.
- **Open Source**: Being open-source means students can freely access Python and its libraries, encouraging experimentation and collaboration.

5. Industry Demand

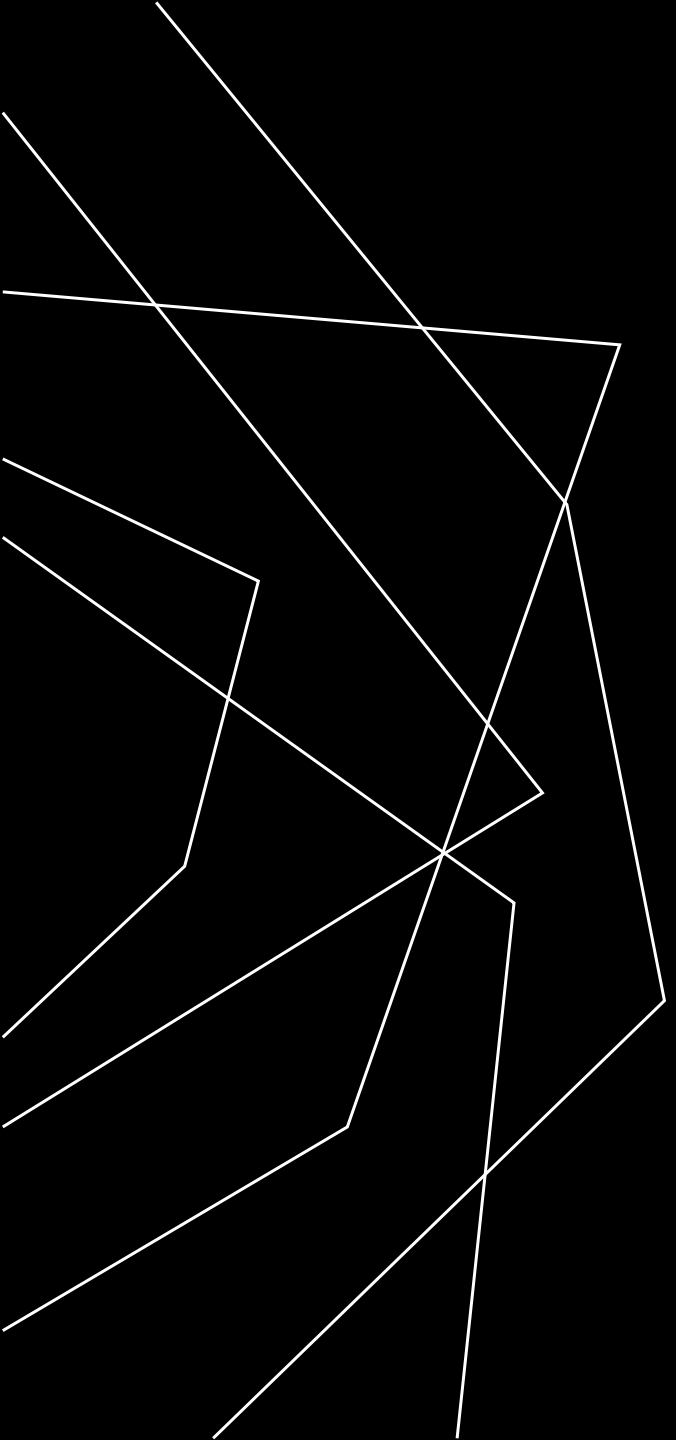
- **High Demand for Python Skills**: Many companies in AI, data science, and tech industries look for Python proficiency in candidates.
- **Career Opportunities**: Knowledge of Python opens up various career paths, including data analyst, machine learning engineer, and AI researcher.

6. Integration with Other Technologies

- **Interoperability**: Python can easily integrate with languages like C, C++, and Java, making it suitable for projects that require high performance.
- **APIs and Web Services**: Python can be used to create RESTful APIs, facilitating the development of web applications that leverage AI.

7. Focus on AI/ML Concepts

- **Learning Focus**: Python allows students to focus on understanding AI and machine learning concepts without getting bogged down by complex syntax.
- **Experimentation**: The interactive nature of Python (especially with tools like Jupyter Notebooks) encourages hands-on learning and experimentation.



THANK YOU